

Figure 3: Exynos 5420 SoC comprising heterogeneous CPUs (big.LITTLE architecture) [Source: <http://www.cnx-software.com/2013/11/08/179-arndale-octa-development-board-gets-an-upgrade-to-exynos-5420-big-lit-tle-soc/>]

enhancement, and was later summarised as the well-known Amdahl's Law. When we apply Amdahl's Law to parallel processing, we have the speedup metric as:

$$S_{\text{Amdahl}} = \frac{\text{Performance}_{\text{new}}}{\text{Performance}_{\text{old}}} = \frac{\text{SequentialExecutionTime}}{\text{ParallelExecutionTime}} = \frac{T_s}{T_p}$$

Let's suppose that α is the fraction of the code that is sequential, which cannot be parallelised, and p is the number of processors. Assuming that all overheads are ignored, we have:

$$S_{\text{Amdahl}} = \frac{T_s}{T_p} = \frac{T_s}{\alpha T_s + (1 - \alpha) T_s / p} = \frac{1}{\alpha + (1 - \alpha) / p}$$

This formula is called Amdahl's Law for parallel processing. When p , the number of processors, increases to infinity, the speed-up becomes $\lim_{p \rightarrow \infty} S_{\text{Amdahl}} = \lim_{p \rightarrow \infty} 1 / (\alpha + (1 - \alpha) / p) = 1 / \alpha$. This equation shows that the speed-up is limited by the sequential fraction, a nature of the problem under study, even when the number of processors is scaled to infinity. Amdahl's Law advocates that large-scale parallel processing is less interesting because the speed-up has an upper boundary of $1/\alpha$.

GPGPU

As the multi-core era was evolving, some adventurous programmers were exploring ways to leverage other compute units like the GPU (graphics processing unit) for general purpose computing. The concept of the GPGPU (general purpose graphic processing units) evolved at NVIDIA, AMD and other organisations. This involved offloading some pieces of code so that it runs on the GPU in parallel and boosts performance significantly. With the multi-core (scale-up model) era hitting a dead end, heterogeneous system architecture did show a ray of hope (scale-out model) by getting all the various compute units like the GPU, DSPs, ASSPs and FPGA together to address the current day's needs.

Heterogeneous systems

Heterogeneous system architecture integrates heterogeneous

processing elements into a coherent processing environment enabling power-efficient performance. It is designed to enable extremely flexible, efficient processing for specific workloads and increased portability of code across processors and platforms. Figure 4 shows a hardware vendor's perspective.

There are a wide range of applications that need heterogeneous high performance computing (HPC), such as astrophysics, atmospheric and ocean modelling, bio-informatics, bio-molecular simulation, protein folding, computational chemistry, computational fluid dynamics, computational physics, computer vision and image understanding, data mining and data-intensive computing, global climate modelling and forecasting, material sciences and quantum chemistry. Figure 5 provides an overview of AMD's heterogeneous system architecture, called HSA.

So what does a heterogeneous system comprise? Dissecting such a system reveals varying types of computing units, interconnects, memory and software entities. The different types of compute units are the CPU, GPU, ASIC, DSP, ASSPs, FPGAs and SoCs. Various interconnects such as fabric connectivity, InfiniBand, RapidIO, PCIe, Ethernet and Omni-path are used to connect all these compute units. The memory hierarchies of each of the compute units access these memories in a unified manner. The software entities are largely impacted to support HSA, including the operating system, virtual machine (ISA), runtime libraries and the compiler tool chain.

Heterogeneous systems pose a unique challenge in computing as they contain distinct sets of compute units with their very own architecture. Since the instruction set architecture of each of the compute units is different, it becomes difficult to achieve load balancing between them. Each compute unit views the memory spaces disjointedly and hence consistency and coherency semantics are a challenge. Memory bandwidth and data transfer between each of the compute units can be a challenge too. Each of these units has its own programming models.

A CPU-centric execution model requires all functions to be delegated by the CPU even if intended for an accelerator.

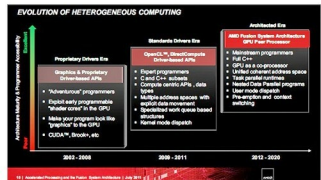


Figure 4: The various eras of heterogeneous systems [Source: <http://semiweekly.com/2011/06/20/amd-talks-about-next-generation-software-and-fusion/>]